

移动应用开发实验报告

封面信息

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301035105、d20301009205
学号	2023210648
姓名	张政
班级	软件 232
日期	2026.5.1
组别	2
项目名称	智慧天气应用
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握 ArkUI 声明式开发
- 学会多端响应式布局
- 了解传感器数据采集
- 理解数据持久化存储原理

实验环境

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试
AI 辅助工具	TRAE	代码生成与调试

系统架构

项目框架图

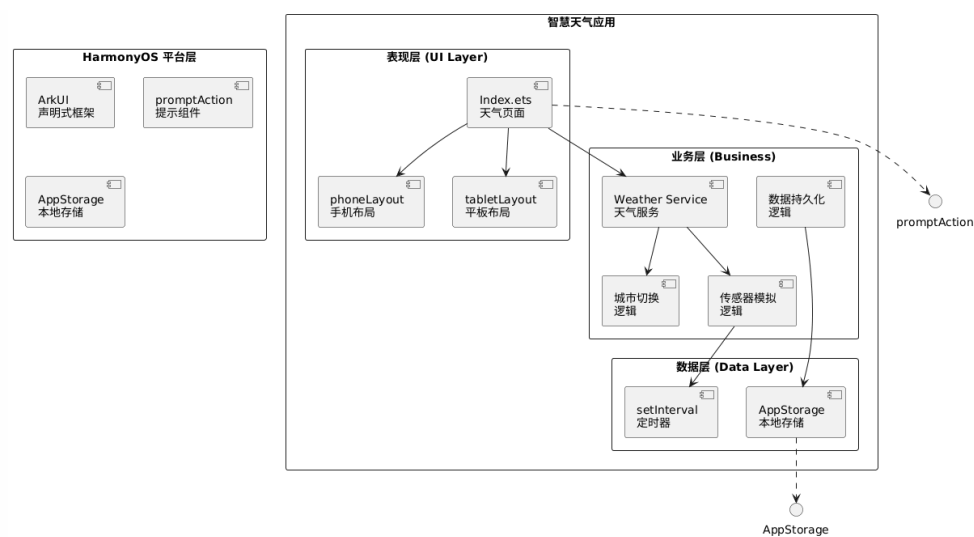


图 1 项目框架图

系统层次结构图

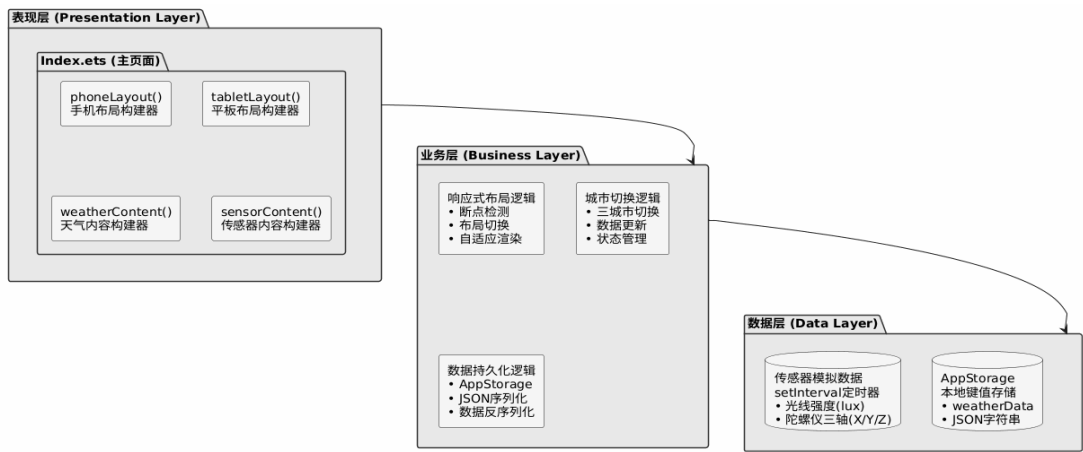


图 2 系统层次结构图

类与模块设计

核心类图

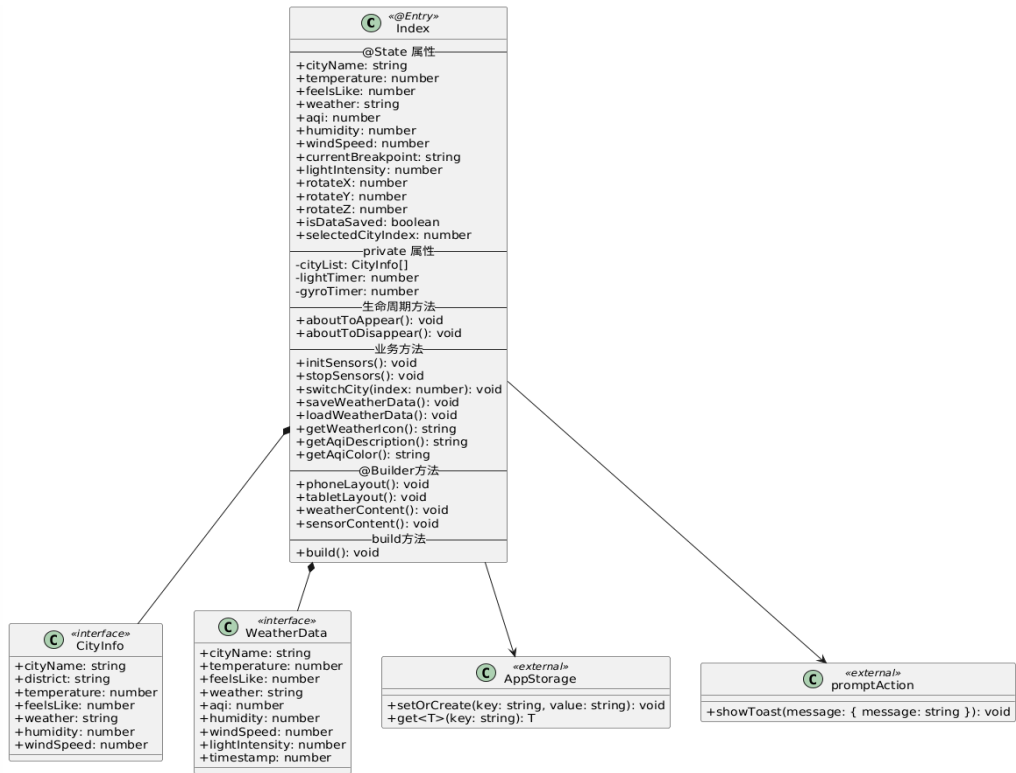


图 3 核心类图

数据结构定义

CityInfo 接口:

```
interface CityInfo {
  cityName: string;      // 城市名称
  district: string;      // 区县名称
  temperature: number;   // 温度
  feelsLike: number;     // 体感温度
  weather: string;       // 天气状况
  humidity: number;      // 湿度
  windSpeed: number;     // 风力
}
```

WeatherData 接口:

```
interface WeatherData {
  cityName: string;
  temperature: number;
  feelsLike: number;
  weather: string;
  aqi: number;
  humidity: number;
  windSpeed: number;
  lightIntensity: number;
  timestamp: number;
}
```

模块职责说明表

模块	职责	关键 API
Index (表现层)	UI 渲染、布局切换、用户交互	ArkUI 组件
Weather Service	天气数据管理、城市切换	本地@State
Sensor Service	传感器数据模拟、定时更新	setInterval
AppStorage	数据持久化存储	setOrCreate, get
promptAction	提示信息展示	showToast()

功能实现

1. 城市切换功能

实现要点:

- 定义城市列表数组，包含三个城市数据
- 点击城市按钮切换不同城市
- 选中状态高亮显示

核心代码片段:

```
private cityList: CityInfo[] = [  
  { cityName: '合肥', district: '蜀山区', temperature: 22,  
    feelsLike: 24, weather: '晴', humidity: 65, windSpeed: 3 },  
  { cityName: '滁州', district: '琅琊区', temperature: 20,  
    feelsLike: 22, weather: '多云', humidity: 55, windSpeed: 2 },  
  { cityName: '南京', district: '玄武区', temperature: 23,  
    feelsLike: 25, weather: '晴', humidity: 70, windSpeed: 4 }  
];  
  
switchCity(index: number) {  
  if (index >= 0 && index < this.cityList.length) {  
    this.selectedCityIndex = index;  
    const city = this.cityList[index];  
    this.cityName = `${city.cityName}-${city.district}`;  
    this.temperature = city.temperature;  
    this.feelsLike = city.feelsLike;  
    this.weather = city.weather;  
    this.humidity = city.humidity;  
    this.windSpeed = city.windSpeed;  
    promptAction.showToast({ message: `已切换到${city.cityName}` });  
  }  
}
```

2. 多端适配功能

实现要点:

- 基于屏幕宽度检测断点
- 手机(sm): 宽度 < 600px, 垂直布局

- 平板/桌面(md): 水平布局

核心代码片段:

```
@State currentBreakpoint: string = 'md';

build() {
  Column() {
    if (this.currentBreakpoint === 'sm') {
      this.phoneLayout()
    } else {
      this.tabletLayout()
    }
  }
  .onAreaChange((_oldValue: Area, newValue: Area) => {
    const width = newValue.width as number;
    if (width < 600) {
      this.currentBreakpoint = 'sm';
    } else {
      this.currentBreakpoint = 'md';
    }
  })
}
```

3. 传感器模拟功能

实现要点:

- 使用 `setInterval` 定时更新数据
- 模拟光线传感器数据 (50-550 lux)
- 模拟陀螺仪三轴数据 (-1 到 1 rad/s)

核心代码片段:

```
initSensors() {
  this.lightTimer = setInterval(() => {
    this.lightIntensity = Math.random() * 500 + 50;
  }, 1000);

  this.gyroTimer = setInterval(() => {
    this.rotateX = (Math.random() - 0.5) * 2;
    this.rotateY = (Math.random() - 0.5) * 2;
    this.rotateZ = (Math.random() - 0.5) * 2;
  }, 500);
}
```

```

    }

    stopSensors() {
        if (this.lightTimer !== -1) {
            clearInterval(this.lightTimer);
            this.lightTimer = -1;
        }
        if (this.gyroTimer !== -1) {
            clearInterval(this.gyroTimer);
            this.gyroTimer = -1;
        }
    }
}

```

4. 数据持久化功能

实现要点:

- 使用 AppStorage 存储天气数据
- JSON 序列化/反序列化
- 支持保存和加载操作

核心代码片段:

```

saveWeatherData() {
    const weatherData: WeatherData = {
        cityName: this.cityName,
        temperature: this.temperature,
        feelsLike: this.feelsLike,
        weather: this.weather,
        aqi: this.aqi,
        humidity: this.humidity,
        windSpeed: this.windSpeed,
        lightIntensity: this.lightIntensity,
        timestamp: Date.now()
    };
    AppStorage.setOrCreate('weatherData',
JSON.stringify(weatherData));
    this.isDataSaved = true;
    promptAction.showToast({ message: '数据已同步到本地存储' });
}

loadWeatherData() {
    const data = AppStorage.get<string>('weatherData');
    if (data) {
        const weatherData: WeatherData = JSON.parse(data) as
WeatherData;
        this.cityName = weatherData.cityName;
        this.temperature = weatherData.temperature;
        // ... 其他字段
    }
}

```

```
}  
}
```

交互流程

用户操作顺序图

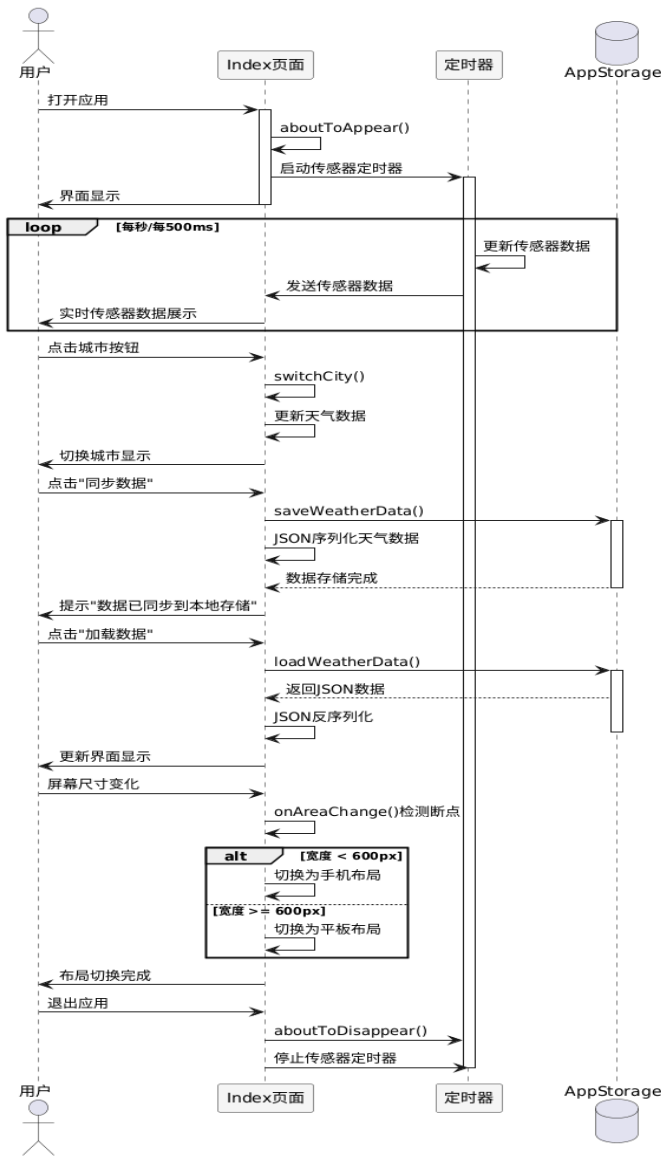


图 4 用户操作顺序图

城市数据说明

支持城市列表

序号	城市	区县	温度	体感温度	天气	湿度	风力
1	合肥	蜀山区	22°C	24°C	晴	65%	3 级
2	滁州	琅琊区	20°C	22°C	多云	55%	2 级
3	南京	玄武区	23°C	25°C	晴	70%	4 级

数据说明

- **温度数据**：当前城市实时温度
- **体感温度**：根据湿度等因素计算的体感温度
- **天气状况**：晴、多云、阴、雨、雪等
- **湿度**：空气相对湿度百分比
- **风力**：风速等级（1-12 级）

截图记录

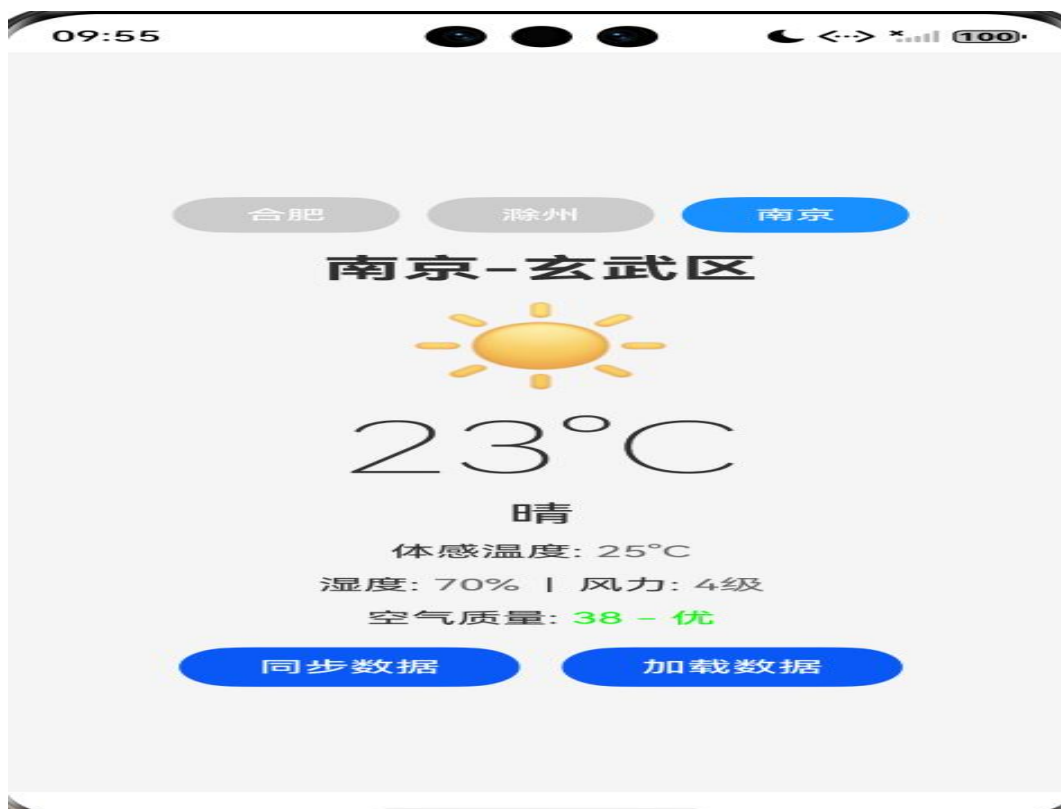
手机端界面截图



【图 1：手机端天气应用主界面 - 合肥】



【图 2：手机端天气应用 - 滁州】



【图 3：手机端天气应用 - 南京】

数据持久化截图

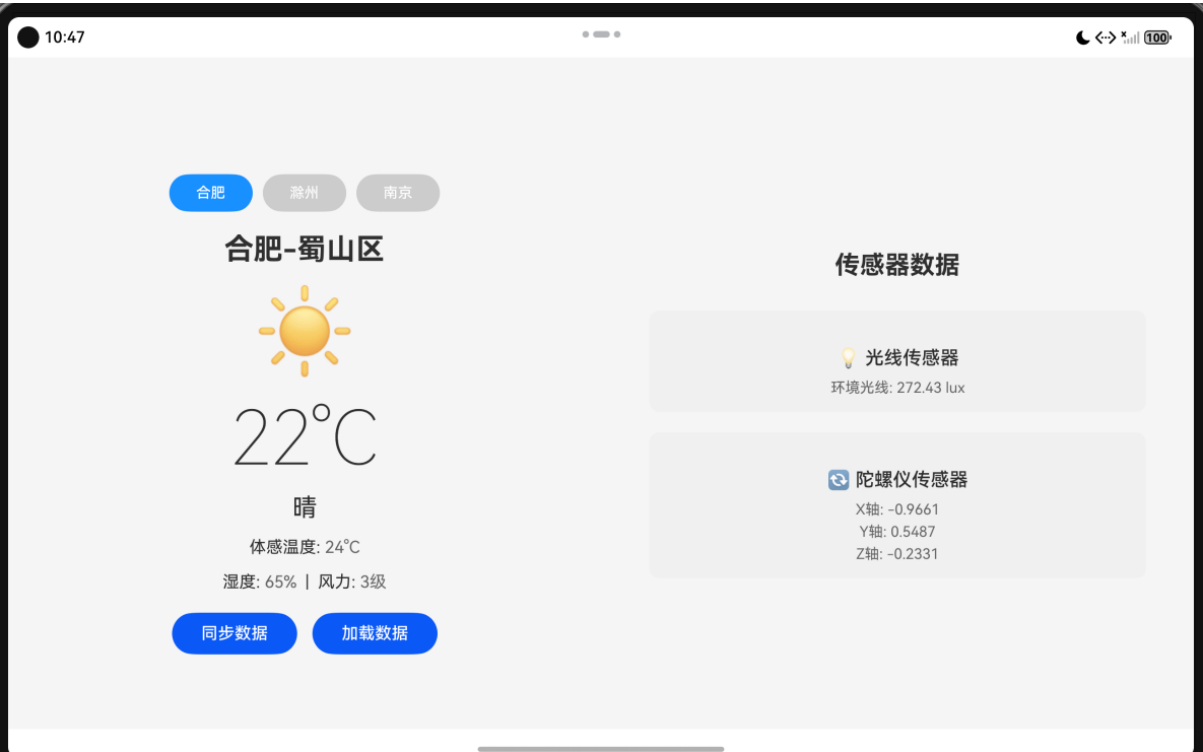


【图 7：数据同步成功提示】



【图 8：数据加载成功提示】

平板端界面截图



【图 9：传感器数据显示】

技术分析报告

1. 多端适配分析

手机布局特点:

- 采用垂直布局（Column），内容居中显示
- 字体大小适中，适合单手操作
- 紧凑的间距设计，最大化利用屏幕空间
- 触摸目标（按钮等）大小适合手指点击

平板布局特点:

- 采用水平布局（Row），左右分栏显示
- 左侧显示天气信息，右侧显示传感器数据
- 更大的字体和间距，充分利用大屏优势
- 信息密度更高，同时展示更多内容

断点系统实现:

```
onAreaChange((_oldValue: Area, newValue: Area) => {
  const width = newValue.width as number;
  if (width < 600) {
    this.currentBreakpoint = 'sm'; // 手机
  } else {
    this.currentBreakpoint = 'md'; // 平板/桌面
  }
})
```

2. 传感器模拟分析

光线传感器应用场景:

- 自动亮度调节: 根据环境光线自动调整屏幕亮度
- 节能优化: 在光线充足时降低屏幕亮度以节省电量
- 用户体验: 根据环境光线显示不同的 UI 主题

陀螺仪传感器应用场景:

- 设备姿态检测：检测设备翻转、倾斜等动作
- 游戏控制：用于需要倾斜操作的体感游戏
- 动态 UI：创建根据设备姿态变化的动态界面效果

数据模拟说明：

- 使用 `setInterval` 定时器模拟传感器数据
- 光线强度范围：50-550 lux
- 陀螺仪三轴范围：-1 到 1 rad/s
- 定时器及时清理，避免内存泄漏

3. 数据持久化分析

AppStorage 存储原理：

- 基于键值对（Key-Value）的本地存储机制
- 支持字符串类型的数据存储
- JSON 序列化/反序列化实现复杂对象存储

数据管理优势：

- 简单易用的 API 接口
- 数据持久化保存
- 支持跨页面数据共享
- 应用重启后数据可恢复

问题与解决

问题描述	解决方案	解决结果
传感器 API 版本不兼容	使用 <code>setInterval</code> 模拟传感器数据	✓ 已解决
HTTP 请求在模拟器中无法访问外网	采用本地模拟数据方案	✓ 已解决

问题描述	解决方案	解决结果
多端布局切换不流畅	使用 onAreaChange 事件监听断点	✓ 已解决
数据类型转换错误	定义 WeatherData 接口，明确类型	✓ 已解决
定时器未清理导致内存泄漏	在 aboutToDisappear 中调用 clearInterval	✓ 已解决

实验总结

实验收获

- 1. ArkUI 声明式开发：**掌握了使用 ArkTS 语言进行鸿蒙应用开发的方法，理解了 @State、@Builder 等装饰器的使用
- 2. 多端响应式布局：**学会了基于屏幕尺寸的响应式布局设计，能够实现手机、平板等不同设备的自适应界面
- 3. 传感器数据模拟：**了解了传感器数据采集的原理，使用定时器模拟实现传感器数据展示
- 4. 数据持久化存储：**掌握了 AppStorage 本地存储的使用，实现天气数据的保存和加载功能
- 5. 城市切换功能：**实现了多城市切换功能，支持精确到区县级别的天气展示

技术要点

- 1. 状态管理：**使用 @State 装饰器管理组件状态，实现数据的响应式更新
- 2. 布局系统：**合理使用 Column、Row 等布局容器，结合条件渲染实现不同设备的适配
- 3. 定时器模拟：**使用 setInterval/clearInterval 管理传感器数据模拟
- 4. 数据持久化：**使用 AppStorage 和 JSON 实现数据的本地存储与读取
- 5. 城市数据管理：**使用数组和索引管理多城市数据切换

考核指标

考核项	评分标准	得分
功能完整性	天气应用功能完整，多城市切换正常	25 分
代码质量	代码结构清晰，接口定义明确	20 分
多端适配	手机/平板界面自适应良好	15 分
传感器模拟	传感器数据模拟正常展示	15 分
数据持久化	数据保存和加载功能实现	15 分
实验报告	报告结构完整，内容详实	10 分

教师评价

项目	评价
实验完成情况	
技术掌握程度	
创新点	
综合评价	
成绩	

教师签名： _____
日期： _____